Advanced Algorithms

Lecture Notes

Lucas Imonetti

2025–2026

Contents

1 Introduction & Exact Pattern Matching **5**
1.1 Definition 5
1.2 Naive approach 5
1.3 Knuth-Morris-Pratt (KMP) algorithm 6
1.3.1 Computing the failure function 7

Chapter 1

Introduction & Exact Pattern Matching

1.1 Definition

The exact pattern matching problem is defined as follows:

Problem 1.1.1 (Exact pattern matching)**.** Given an alphabet Σ, and two strings T, $P \in \Sigma^*$, for which $|T| = n$ and $|P| = m$, find all occurrences of P in T.

Example 1.1.2. Let $\Sigma = \{a, b\}$, $T =$ `ababbab` and $P =$ `abb`. The occurrences of P in T are at positions 2 and 4.

1.2 Naive approach

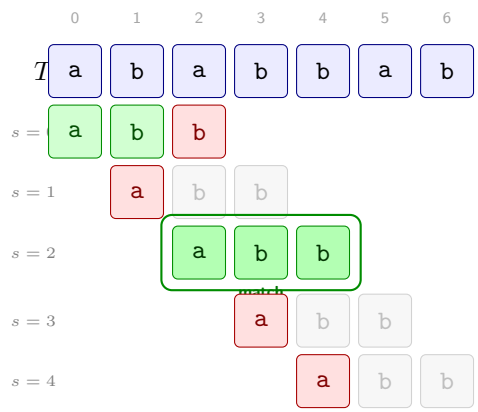


Figure 1.1: Naive exact pattern matching of $P =$ `abb` in $T =$ `ababbab` (0-indexed). Each row corresponds to one shift s; the pattern P is aligned starting at $T[s]$. **Green** = character matched, **red** = first mismatch (algorithm skips to next shift), gray = not compared.

What the Figure 1.1 illustrates is one basic naive approach. In this approach we nest two loops: the outer loop iterates over all possible "first positions" s in T where the pattern P could potentially match, and the inner loop checks character by character if P matches T starting from position s. For example the first row of the figure corresponds to starting the search from index $s = 0$ of T. Given s we then loop one by one each character of P, moving along T as

well, and check if all the characters match. In this case we find a mismatch at the third character of P (which is 'b', whereas the corresponding character in T is 'a'). When this happens, we break this inner loop and move to the next shift $s = 1$. If the lenght of the pattern was m and the length of the text was n, the first outer iterations could take up to $O(m)$ time, and we could have up to $O(n)$ such iterations, leading to a worst-case time complexity of $O(nm)$ for this naive approach.

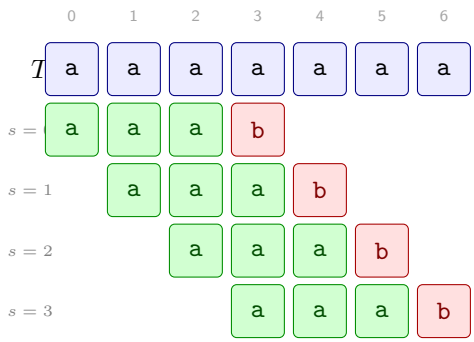


Figure 1.2: Worst-case input for the naive algorithm: $T = \texttt{aaaaaaa}$ ($n = 7$) and $P = \texttt{aaab}$ ($m = 4$). Every shift s performs $m - 1$ successful comparisons before the inevitable mismatch on $P[m - 1] = \texttt{b}$, giving $(n - m + 1)(m - 1) = \Theta(nm)$ total comparisons.

In figure 1.2 we see an example of a worst-case input for the naive algorithm. In this case the text T consists of n 'a' characters, and the pattern P consists of $m-1$ 'a' characters followed by a 'b'. There are $n-m+1$ possible shifts s where the pattern could potentially match, and for each of these shifts the algorithm will successfully match the first $m - 1$ characters of P. In the example of the figure we have $n = 7$ and $m = 4$, so there are $7 - 4 + 1 = 4$ shifts to check, and for each shift we make m checks, of which $m - 1 = 3$ are successful matches before we encounter the mismatch at the last character of P. This results in a total of $(n - m + 1)(m - 1)$ comparisons, which is $\Theta(nm)$ in the worst case.

This naive approach basically starts over from scratch at each shift, without leveraging precious information from previous comparisons.

1.3 Knuth-Morris-Pratt (KMP) algorithm

What if we could somehow "remember" the information from a previous comparison, and use it to skip some of the checks at the next shift? Let's introduce the intuition behind the Knuth-Morris-Pratt (KMP) algorithm, with an example.

Example 1.3.1. Let $P = \texttt{abcdeabdef}$ and suppose we are running the naive algorithm at shift $s = 2$ over some text T whose characters at positions 2–8 are `abcdeab`. We match $P[0 \ldots 6]$ successfully, then hit a mismatch at $P[7] = \texttt{d}$ vs $T[9] = \texttt{x}$.

As shown in Figure 1.3, at this point the naive algorithm would discard everything and try shift $s = 3$. But notice that the matched portion `abcdeab` starts *and* ends with $\alpha = \texttt{ab}$: the prefix $P[0 \ldots 1]$ and the suffix $P[5 \ldots 6]$ of the matched region are identical. This means that when we slide P forward, the α that was at the end of the match is now perfectly aligned with the α at

the start of P — we can jump directly to shift $s = 7$ and resume comparing from $P[2]$, skipping five shifts entirely.

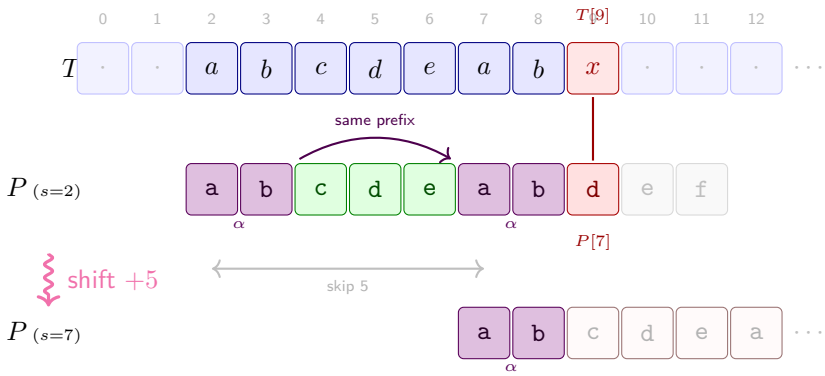
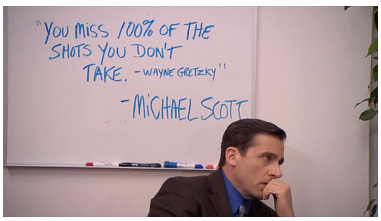


Figure 1.3: KMP shift intuition with $P =$ `abcdeabdef`. At shift $s = 2$, the algorithm matches $P[0 \ldots 6] =$ `abcdeab` then hits a mismatch at $P[7]$. The matched prefix ends with $\alpha =$ `ab`, which also appears at the very start of P — so instead of shifting by 1, we jump directly to $s = 7$, re-aligning α and resuming from $P[2]$.

Basically what the example shows is that if we take the longest prefix of P that is also a suffix of the **matched** portion of P, we can shift the pattern so that this prefix is now aligned with the suffix — and we can resume the search from the character right after this prefix, since we know it must match (otherwise we would have had a mismatch earlier). In other words we have saved some works (namely, three shift indexes) by leveraging the information from the previous comparisons, instead of starting over from scratch, from the very next shift.

Important: when we say suffix we mean suffix of the ***matched*** *portion of P, not necessarily a suffix of the whole pattern. In the example, α is a suffix of $P[0 \ldots 6]$ but not of P itself.*

But is this correct? Or do we miss some potential matches by jumping directly to $s = 7$?



"You miss 100% of the shots you don't take." — Michael Scott

Well, it turns out that this is indeed correct, and we won't miss any matches by doing this. The KMP algorithm is based on this very idea, and it guarantees that we will find all occurrences of P in T without missing any, while also achieving a much better time complexity than the naive approach.

1.3.1 Computing the failure function

To exploit this idea, we need to precompute for every prefix $P[0 \ldots j]$ its *longest proper prefix that is also a suffix*, written $\pi[j]$. This table is known as the **failure function** (or prefix function) of P.

A proper *prefix/suffix excludes the whole string itself.*

Definition 1.3.2 (Failure function)**.** Let P be a pattern of length m. The

failure function $\pi : \{0, \ldots, m-1\} \to \{0, \ldots, m-1\}$ is defined as

$$\pi[j] \;=\; \max\{\, k < j+1 \mid P[0 \ldots k-1] = P[j-k+1 \ldots j] \,\},$$

i.e. the length of the longest proper prefix of $P[0 \ldots j]$ that is also a suffix of $P[0 \ldots j]$.

The key observation is that $\pi[j]$ can be computed recursively from $\pi[j-1]$, giving an efficient $\mathcal{O}(m)$ preprocessing algorithm. There are two cases:

1. **Extension is possible.** If $P[\pi[j-1]] = P[j]$, then the longest prefix-suffix of $P[0 \ldots j]$ is simply one character longer than that of $P[0 \ldots j-1]$, so $\pi[j] = \pi[j-1] + 1$.

   Example: with $P =$ `abcdeabdef` at $j = 6$, we have $\pi[5] = 1$ (prefix-suffix of `abcdea` is `a`). We check $P[1] =$ `b` against $P[6] =$ `b`: they match, so $\pi[6] = 2$.

2. **Extension is not possible.** If $P[\pi[j-1]] \neq P[j]$, we cannot extend the current prefix-suffix. We then ask: what is the longest prefix-suffix of the prefix-suffix itself? In other words, we fall back to $\pi[\pi[j-1]-1]$ and try to extend that instead. We repeat this process until either a match is found or we reach length 0, in which case $\pi[j] = 0$.

   Example: still with $P =$ `abcdeabdef` at $j = 7$, we have $\pi[6] = 2$ (prefix-suffix of `abcdeab` is `ab`). We check $P[2] =$ `c` against $P[7] =$ `d`: no match. We fall back to $\pi[1] = 0$, so we try extending a prefix-suffix of length 0: we check $P[0] =$ `a` against $P[7] =$ `d`, still no match. We have reached length 0, so $\pi[7] = 0$.

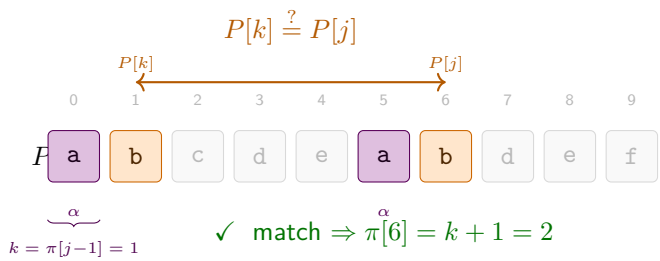


Figure 1.4: Computing $\pi[6]$ for $P =$ `abcdeabdef`. We know $k = \pi[5] = 1$: the suffix of $P[0 \ldots 5]$ that matches a prefix of P has length 1 (namely $\alpha =$ `a`, violet). We then compare $P[k] = P[1] =$ `b` against $P[j] = P[6] =$ `b` (orange): they match, so we extend and set $\pi[6] = 2$.

The resulting algorithm is the following:

Algorithm 1: KMP failure function

```
Input: Pattern P[0 ... m − 1]
Output: Failure function π[0 ... m − 1]
π[0] ← 0
k ← 0            // length of current candidate prefix-suffix
for j ← 1 to m − 1 do
    while k > 0 and P[k] ≠ P[j] do
        k ← π[k − 1]                                  // fall back
    if P[k] = P[j] then
        k ← k + 1
    π[j] ← k
return π
```

Remark 1.3.3. The "fall back to the prefix-suffix of the prefix-suffix" is the recursive heart of KMP: the same shift logic used during the search phase is also used during preprocessing.

Exercise 1.3.4. Design an algorithm that, given a pattern P of length m, computes $\pi[i]$ for every $i \in \{0, \ldots, m-1\}$ in $\mathcal{O}(m)$ time.

Hints:

1. Try to express $\pi[i]$ in terms of $\pi[i-1]$. What is the relationship between the longest prefix-suffix of $P[0 \ldots i]$ and that of $P[0 \ldots i-1]$?

2. There are two cases depending on whether $P[\pi[i-1]]$ equals $P[i]$ or not. What can you do in each case?

3. For the harder case, think recursively: if you cannot extend a prefix-suffix of length k, which shorter candidate should you try next?